Лабораторная работа №6

Декораторы, контекстные менеджеры и расширенные возможности Python

1. Цель работы

Освоить dataclass, Enum, аннотации, декораторы и functools.wraps, контекстные менеджеры, Protocol и интроспекцию на примере сервиса доставок.

2. Постановка индивидуального задания

Вариант 12 — «Курьерская служба». Реализовать сервис обработки доставок с расширенными конструкциями Python.

Программа должна:

– Создать модели курьера, посылки и доставки с пятью состояниями: создана, принята, в пути, доставлена, отменена.

– Предусмотреть добавление, поиск, удаление, смену статуса, расчёт стоимости и средней длительности.

– Использовать декораторы, менеджер контекста, Protocol и интроспекцию.

3. Краткие теоретические сведения

@dataclass автоматически создаёт стандартные методы хранения и представления данных, а Enum задаёт фиксированный набор допустимых состояний. Аннотации типов делают интерфейсы понятнее, но не заменяют проверку входных данных.

Декоратор оборачивает функцию, добавляя поведение до или после вызова. @wraps сохраняет метаданные исходной функции. В проекте @log сообщает о вызове, @timed измеряет длительность, а @require_status проверяет допустимость завершения доставки.

Контекстный менеджер operation() построен на @contextmanager и гарантированно выводит сведения о завершении в finally. Protocol описывает

ожидаемый метод report(). Интроспекция через type(), isinstance(), hasattr(), getattr() и dir() позволяет изучать объект во время работы.

4. Исходный код программы

Файл models.py

```
from dataclasses import dataclass
from enum import Enum


class Status(Enum):
    CREATED = "создана"
    ACCEPTED = "принята"
    MOVING = "в пути"
    DELIVERED = "доставлена"
    CANCELLED = "отменена"


@dataclass
class Entity:
    id: int
    name: str

    def report(self) -> str:
        return f"{self.id}: {self.name}"


@dataclass
class Courier(Entity):
    phone: str

    def report(self) -> str:
        return f"Курьер {self.name}, телефон {self.phone}"


@dataclass
class Parcel(Entity):
    weight: float

    def report(self) -> str:
        return f"Посылка {self.name}, {self.weight} кг"


@dataclass
class Delivery:
    id: int
    courier: Courier
    parcel: Parcel
    distance: float
    duration: float = 0
    status: Status = Status.CREATED
```

Файл decorators.py

```
from functools import wraps
from time import perf_counter
from models import Status
```

```
def log(function):
    @wraps(function)
    def wrapper(*args, **kwargs):
        print("Вызов:", function.__name__)
        return function(*args, **kwargs)
    return wrapper


def timed(function):
    @wraps(function)
    def wrapper(*args, **kwargs):
        start = perf_counter()
        result = function(*args, **kwargs)
        print(f"Время {function.__name__}: {perf_counter() - start:.6f} c")
        return result
    return wrapper


def require_status(status: Status):
    def decorate(function):
        @wraps(function)
        def wrapper(self, delivery, *args, **kwargs):
            if delivery.status != status:
                raise ValueError("Недопустимый статус для операции")
            return function(self, delivery, *args, **kwargs)
        return wrapper
    return decorate
```

Файл helpers.py

```
from contextlib import contextmanager
from time import perf_counter
from typing import Protocol
from models import Entity


class Reportable(Protocol):
    def report(self) -> str: ...


def show_report(item: Reportable) -> None:
    print(item.report())


@contextmanager
def operation(name: str):
    start = perf_counter()
    print("Начало:", name)
    try:
        yield
    finally:
        print(f"Конец: {name}, {perf_counter() - start:.6f} c")


def inspect_object(item: object) -> None:
    print("Тип:", type(item).__name__)
```

```
    print("Это Entity:", isinstance(item, Entity))
    print("Есть id:", hasattr(item, "id"))
    print("Значение id:", getattr(item, "id", None))
    print("Публичные имена:", [name for name in dir(item) if not name.startswith("_")])
```

Файл services.py

```
from models import Delivery, Status
from decorators import log, timed, require_status


class DeliveryNotFound(Exception):
    pass


class DeliveryService:
    def __init__(self) -> None:
        self.deliveries: list[Delivery] = []

    @log
    def add(self, delivery: Delivery) -> None:
        if delivery.id <= 0 or delivery.distance < 0 or delivery.parcel.weight <= 0:
            raise ValueError("Неверные параметры доставки")
        if any(d.id == delivery.id for d in self.deliveries):
            raise ValueError("Этот id уже существует")
        self.deliveries.append(delivery)

    def find(self, id: int) -> Delivery:
        for delivery in self.deliveries:
            if delivery.id == id:
                return delivery
        raise DeliveryNotFound("Доставка не найдена")

    def delete(self, id: int) -> None:
        self.deliveries.remove(self.find(id))

    def change_status(self, id: int, status: Status) -> None:
        delivery = self.find(id)
        allowed = {Status.CREATED: [Status.ACCEPTED, Status.CANCELLED],
                   Status.ACCEPTED: [Status.MOVING, Status.CANCELLED],
                   Status.MOVING: [Status.DELIVERED, Status.CANCELLED]}
        if status not in allowed.get(delivery.status, []):
            raise ValueError("Недопустимый переход статуса")
        if status == Status.DELIVERED:
            raise ValueError("Используйте пункт завершения доставки")
        delivery.status = status

    @require_status(Status.MOVING)
    def finish(self, delivery: Delivery, duration: float) -> None:
        if duration <= 0:
            raise ValueError("Продолжительность должна быть положительной")
        delivery.duration = duration
        delivery.status = Status.DELIVERED

    def cost(self, delivery: Delivery) -> float:
        return 100 + delivery.distance * 20 + delivery.parcel.weight * 10
```

```
    @timed
    def statistics(self) -> dict:
        completed = [d for d in self.deliveries if d.status == Status.DELIVERED]
        average = sum(d.duration for d in completed) / len(completed) if completed else 0
        return {"всего": len(self.deliveries), "доставлено": len(completed),
                "средняя продолжительность, мин": average}
```

Файл main.py

```
from models import Courier, Parcel, Delivery, Status
from services import DeliveryService, DeliveryNotFound
from helpers import operation, show_report, inspect_object


def main() -> None:
    service = DeliveryService()
    courier = Courier(1, "Максим", "9001234567")
    show_report(courier)
    while True:
        print("\n1 Добавить; 2 Удалить; 3 Найти; 4 Все; 5 Статус")
        print("6 Стоимость; 7 Завершить; 8 Статистика; 9 Интроспекция; 10 Демонстрация; 0 Выход")
        choice = input("Действие: ")
        try:
            if choice == "0":
                break
            if choice == "1":
                id = int(input("id: "))
                parcel = Parcel(id, input("Посылка: "), float(input("Вес, кг: ")))
                service.add(Delivery(id, courier, parcel, float(input("Расстояние, км: "))))
            elif choice == "4":
                for delivery in service.deliveries:
                    print(delivery)
            elif choice == "8":
                print(service.statistics())
            elif choice == "10":
                with operation("Демонстрация"):
                    parcel = Parcel(1, "Книга", 1)
                    show_report(parcel)
                    demo = DeliveryService()
                    delivery = Delivery(1, courier, parcel, 5)
                    demo.add(delivery)
                    demo.change_status(1, Status.ACCEPTED)
                    demo.change_status(1, Status.MOVING)
                    demo.finish(delivery, 30)
                    print(demo.statistics())
            elif choice in ("2", "3", "5", "6", "7", "9"):
                delivery = service.find(int(input("id: ")))
                if choice == "2":
                    service.delete(delivery.id)
                elif choice == "3":
                    print(delivery)
                elif choice == "5":
                    print([s.value for s in Status])
                    service.change_status(delivery.id, Status(input("Статус: ")))
                elif choice == "6":
```

```
                print("Стоимость:", service.cost(delivery))
            elif choice == "7":
                service.finish(delivery, float(input("Продолжительность, мин: ")))
            elif choice == "9":
                inspect_object(delivery)
            else:
                print("Неизвестное действие")
        except (ValueError, DeliveryNotFound) as error:
            print("Ошибка:", error)


if __name__ == "__main__":
    main()
```

5. Примеры работы программы в виде снимков экрана

На рисунке 6.1 показаны работа с доставками, измерение длительности операции и сбор статистики.

```
python main.py
Действие: Начало: Демонстрация
Посылка Книга, 1 кг
Вызов: add
Время statistics: 0.000003 с
{'всего': 1, 'доставлено': 1, 'средняя продолжительность, мин': 30.0}
Конец: Демонстрация, 0.000053 с
Действие: id: Посылка: Вес, кг: Расстояние, км: Вызов: add
Действие: id: Стоимость: 260.0
Действие: Время statistics: 0.000003 с
{'всего': 1, 'доставлено': 1, 'средняя продолжительность, мин': 45.0}
Действие: id: Тип: Delivery
Есть id: True
Значение id: 2
```

Рисунок 6.1 – Результаты работы системы доставки

6. Вывод

Реализован сервис доставок с dataclass, Enum, декораторами, контекстным менеджером, Protocol и интроспекцией. Проверены работа переходов статуса, расчёт стоимости и статистика завершённых доставок.

Выполнил	Пахомов Максим	21.10.2026
Принял	Елкин Н. С.	